

Rapport de projet — NodEX (portefeuille crypto)

Auteur : meranville

Dépôt : mobileapp-2026 — branche **meranville**

Chemin du projet dans le dépôt : project/crypto-wallet/

Date du rapport : mars 2026

1. Résumé

J'ai développé **NodEX**, une application mobile de type **portefeuille crypto** avec des fonctions financières (soldes, envoi, historique, virements en euros, carte virtuelle côté API), une **authentification** gérée par **Appwrite**, un **assistant conversationnel** via **Groq**, et des paiements intégrés avec **Stripe** (SDK Flutter). Le tout s'appuie sur une **API REST Laravel** exposée sous **/api**. Les **cours des cryptos** affichés à l'écran proviennent de l'**API publique CoinGecko** (appelée depuis l'app Flutter). Ce document présente le travail réalisé, des **captures d'écran**, l'**architecture**, le **fonctionnement des API**, des **extraits de code expliqués**, ce qui manque pour une **mise à disposition du grand public**, et les **sources** qui m'ont aidé.

2. Contexte et objectifs

- **Contexte** : projet mobile dans le cadre du cours / dépôt partagé L3 (application mobile 2026).
 - **Objectif initial** : disposer d'une application utilisable en **local** (simulateur ou téléphone) connectée à un **backend** documenté.
 - **Périmètre** : prototype avancé / **preuve de concept** orientée fintech, **pas** un produit bancaire agréé.
-

3. Description de la solution

3.1 Vue d'ensemble

Couche	Technologie	Rôle
Mobile	Flutter (Dart)	Interface, navigation, appels HTTP, intégration Appwrite / Stripe
Backend	Laravel (PHP)	API JSON, virements, chat Groq, données carte / portefeuilles côté serveur
Auth	Appwrite	Inscription, connexion, JWT
IA	Groq	Complétions pour l'assistant (modèle type Llama 3.1 instantané)
Cours crypto	CoinGecko (API HTTP)	Prix en EUR et variation 24h (appel depuis
Paielements	Stripe	WalletProvider) Intégration côté app (clé publique, flux prévus dans le projet)

3.2 Arborescence utile

- flutter_app/ — code source Flutter (pubspec.yaml, lib/, scripts run_ios_sim.sh).
- backend-laravel/ — API Laravel (routes/api.php, contrôleurs, migrations, .env.example).
- docs/rapport-captures/ — captures d'écran jointes à ce rapport (fichiers PNG).
- Documentation complémentaire : README.md (racine du dépôt), STRIPE_SETUP.md, MIGRATION_VIREMENTS.md, VIREMENTS_AUDIT.md, etc.

3.3 Dépendances Flutter principales

http, provider, appwrite, flutter_stripe, flutter_secure_storage, local_auth, shared_preferences, crypto, qr_flutter, animations, cupertino_icons.

4. Captures d'écran de l'application

Les images suivantes illustrent les principaux écrans de NodEX (simulateur iPhone, mars 2026). Elles sont stockées dans le dépôt sous docs/rapport-captures/.

4.1 Connexion

Écran d'accueil non connecté : marque **NodEX**, slogan, onglets de méthode de connexion (email, code, SMS, lien), champs email / mot de passe, lien d'inscription. L'authentification réelle passe par **Appwrite**.

4.2 Accueil — tableau de bord

Après connexion : salutation avec le prénom, notifications, carrousel (assistant intelligent, heure locale analogique/numérique), **solde total** en euros, aperçu des portefeuilles (EUR, BTC, etc.), barre d'onglets et bouton flottant **Assistant**.

4.3 Accueil — liste des actifs

Liste des cryptos avec **prix en euros**, variation 24h, solde utilisateur ; bloc **Dernières transactions** (virements reçus / envoyés avec pseudonyme) ; bouton **Assistant**.

4.4 Portefeuilles (« Mes portefeuilles »)

Carte **valeur totale** (crypto + EUR), raccourcis Recevoir / Envoyer / Swap / Acheter / Historique, liste **Vos actifs** avec adresse, cours et actions.

4.5 Historique

Filtres Tout / Envoyés / Reçus / Échanges ; transactions groupées par date ; montants colorés (+ / -) ; navigation par onglets.

4.6 Carte virtuelle

Carte type **VISA** avec gradient, **4 derniers chiffres**, titulaire, date d'expiration ; actions **Bloquer** et **Numéro & CVV** (données sensibles gérées avec précaution côté app).

4.7 Réglages

Profil (initiale, pseudo, email), **Serveur & assistant (IA)**, Ondes, devise EUR, thème sombre, notifications, sécurité (biométrie, déconnexion tous appareils).



FIGURE 1 – Écran de connexion NodEX



FIGURE 2 – Tableau de bord — accueil



FIGURE 3 – Accueil — actifs et dernières transactions



FIGURE 4 – Écran portefeuilles



FIGURE 5 – Historique des transactions

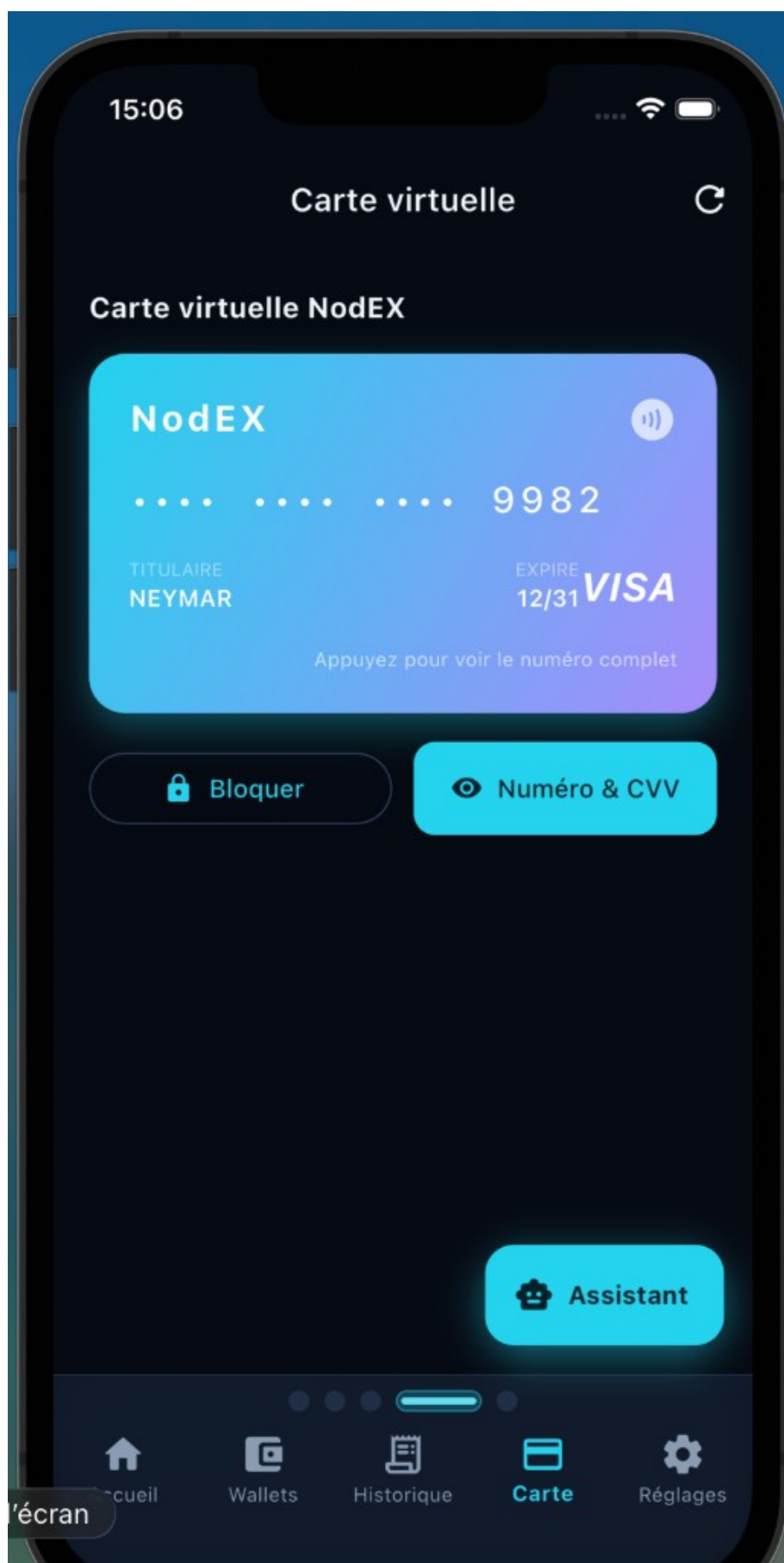


FIGURE 6 – Carte virtuelle

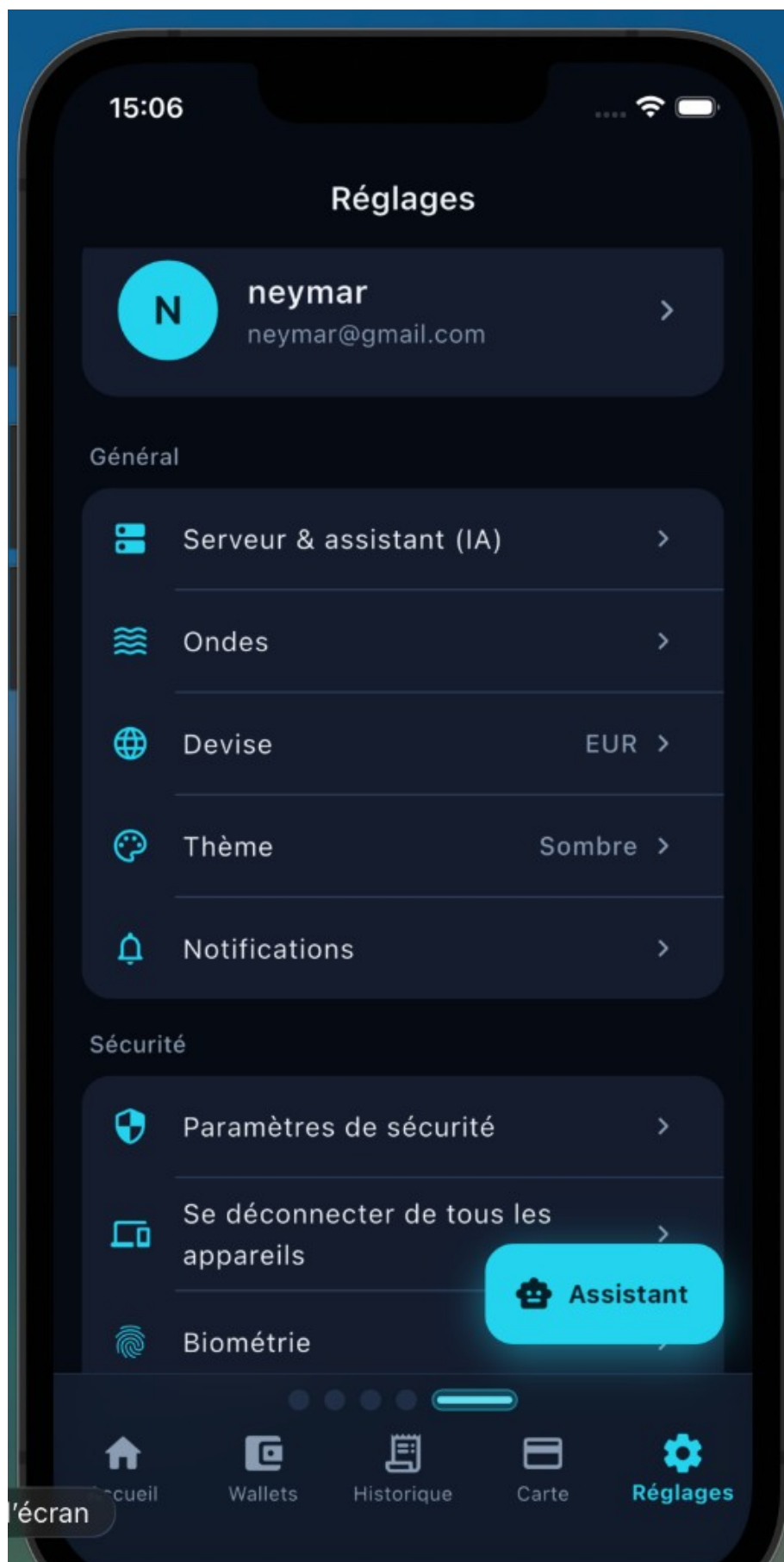


FIGURE 7 – Réglages

4.8 Assistant NodEX (IA)

Conversation avec l'assistant : champ de saisie, envoi vers le backend **ou** secours Groq direct selon configuration (voir section 7).

5. Architecture du projet

5.1 Schéma global

L'application suit une architecture **client** — **serveur** : le mobile Flutter consomme l'API Laravel en JSON, s'authentifie via Appwrite, et appelle des services externes (CoinGecko, Groq, Stripe) soit **depuis le téléphone**, soit **via le serveur** (Groq depuis Laravel pour garder la clé secrète et injecter le contexte compte).

flowchart TB

```
subgraph client [Application Flutter]
    UI[Écrans et widgets]
    AP[AuthProvider / SecurityProvider]
    WP[WalletProvider]
    AC[ApiClient HTTP + JWT]
    AW[SDK Appwrite]
    CH[ChatService]
end
```

end

```
subgraph laravel [Backend Laravel]
    R[routes/api.php]
    WC[WalletController]
    VC[VirementController]
    CC[ChatController]
    CD[CardController]
    DB[(Base de données)]
end
```

end

```
subgraph externe [Services externes]
    APPW[(Appwrite Cloud)]
    CG[CoinGecko API]
    GQ[Groq API]
    ST[Stripe]
end
```

end

UI --> AP

UI --> WP

WP --> AC

WP --> CG

CH --> AC

CH --> GQ

AC --> R

R --> WC

R --> VC

R --> CC

R --> CD

WC --> DB

VC --> DB

CC --> DB

CC --> GQ

CD --> DB

AP --> AW

AW --> APPW

UI --> ST



FIGURE 8 – Assistant NodEX
11

5.2 Navigation dans l'app Flutter

Le fichier `lib/app.dart` définit la logique d'affichage :

1. **Splash** d'ouverture (`NodexOpeningSplash`).
2. Si chargement auth → `LoadingScreen`.
3. Si récupération mot de passe → `UpdatePasswordScreen`.
4. Si email non vérifié → `EmailVerificationPendingScreen`.
5. Si non connecté → `LoginScreen`.
6. Si verrouillage PIN / biométrie → `AppLockScreen`.
7. Sinon → **HomeTabs** : cinq onglets (`DashboardScreen`, `WalletsScreen`, `HistoryScreen`, `CardScreen`, `SettingsScreen`) avec `PageTransitionSwitcher` + `BottomNavigationBar`, fond `FuturisticBackground`, et un **FAB « Assistant »** qui ouvre `ChatScreen`.

5.3 Couches côté Laravel

- **Routes** : `routes/api.php` enregistre les URL sous le préfixe `/api` (souvent monté par `bootstrap/app.php`).
- **Contrôleurs** : logique métier (virements, carte, chat, wallets).
- **Trait ResolvesNodexUser** : lit le JWT Bearer, extrait l'identifiant Appwrite, charge la ligne `NodexUser` en base.
- **Modèles** : `NodexUser`, `VirementEur`, `UserCard`, etc.

6. API REST — liste et fonctionnement

Toutes les URL ci-dessous sont relatives à la **base API** configurée dans l'app, par exemple `http://127.0.0.1:8000/api`. Les routes **protégées** attendent un en-tête :

`Authorization: Bearer <JWT Appwrite>`.

Méthode	Chemin	Rôle
GET	<code>/health</code>	Vérifier que le serveur répond (ok, horodatage).
GET	<code>/debug/auth</code>	Debug uniquement : indique si un Bearer est présent et décode le payload JWT (à supprimer en production).
GET	<code>/wallets</code>	Liste des portefeuilles crypto (ETH, BTC, SOL, ALGO) : symbole, blockchain, adresse dérivée, solde (cache serveur ou 0).
GET	<code>/card</code>	Détail carte virtuelle (création si besoin) : numéro formaté, last4, expiration, CVV, PIN, titulaire.

Méthode	Chemin	Rôle
POST	/chat/groq	Corps JSON : messages , optionnellement model , temperature , max_tokens . Le serveur enrichit le contexte système avec solde EUR, IBAN, pseudonyme, cryptos, derniers virements, puis appelle Groq .
GET	/virements/balance	Solde euro du compte NodEX (balanceEur).
GET	/virements/me	Infos bancaires synthétiques : solde, IBAN, pseudonyme, titulaire.
GET	/virements/history	Historique fusionné envoyés/reçus (limite côté serveur).
POST	/virements/send	Envoi d'un virement interne (montant, destinataire identifié par pseudonyme ou id selon implémentation du contrôleur).

6.1 Chaîne d'authentification JWT → utilisateur Laravel

1. L'utilisateur se connecte avec **Appwrite** dans Flutter.
2. L'app récupère un **JWT** (ou token stocké en secours dans **SharedPreferences**).
3. **ApiClient** ajoute **Authorization: Bearer ...** à chaque requête vers Laravel.
4. Le trait **ResolvesNodexUser** decode le **payload** du JWT (deuxième segment en base64), lit **userId**, **sub** ou champs imbriqués, puis fait **NodexUser::where('appwriteId', ...)->first()**.
5. Si aucune ligne : réponses **401** ou message d'erreur métier.

6.2 Flux assistant IA (Groq)

- **Mode recommandé** : Flutter envoie **POST /api/chat/groq** → Laravel ajoute les **données réelles du compte** dans le message système (pour limiter les inventions du modèle) → Laravel appelle **https://api.groq.com/openai/v1/chat/completions** avec **GROQ_API_KEY** dans **.env**.
- **Mode secours** : si le serveur est indisponible, sans clé, ou erreur réseau, **ChatService** peut appeler Groq **directement** depuis l'app si une clé est saisie dans les réglages (moins idéal pour la confidentialité de la clé).

6.3 Prix des cryptomonnaies

- **Pas** calculés par Laravel dans **WalletController** pour l'affichage marché.
- Le **WalletProvider** (Flutter) appelle **CoinGecko** (**simple/price** en EUR + **include_24hr_change**) pour remplir **_prices** et **_changes24h**, puis combine avec les soldes renvoyés par **GET /api/wallets**.

7. Explication détaillée du code

Cette section décrit à quoi sert chaque partie importante du projet, dans l'ordre logique : démarrage de l'app Flutter, état global, appels réseau, puis chaîne Laravel (middleware → routes → contrôleurs).

7.A Flutter — point d'entrée et thème système

Fichier : flutter_app/lib/main.dart

Élément	Rôle
<code>WidgetsFlutterBinding.ensureInitialized()</code>	Obligatoire avant tout <code>async</code> au démarrage (chargement config, stockage).
<code>usePathUrlStrategy()</code>	Sur le web, les URLs du type <code>/verify-email?...</code> fonctionnent sans <code>#</code> (liens Appwrite).
<code>await ApiConfig.loadFromDisk()</code>	Relit l'URL d'API sauvegardée dans les réglages.
<code>await GroqDirectConfig.loadFromDisk()</code>	Relit une éventuelle clé Groq saisie dans l'app (mode secours).
<code>await StripeService.init()</code>	Initialise le SDK Stripe (clé publique) pour les paiements.
<code>appwriteClient</code>	Accès lazy au client Appwrite : si la config est invalide, l'erreur est loguée puis propagée.
<code>SystemChrome.setSystemUIOverlayStyle</code>	Barre de statut / navigation Android : fond sombre, icônes claires (cohérent avec NodEX).
<code>MultiProvider</code>	Enregistre trois ChangeNotifier globaux : <code>AuthProvider</code> , <code>WalletProvider</code> , <code>SecurityProvider</code> .
<code>AuthProvider()..checkAuth()..initAuthListener()</code>	Au lancement : vérifie si un utilisateur Appwrite est déjà connecté ; prépare le flux auth.
<code>runApp(App())</code>	Affiche le widget racine <code>App</code> (dans <code>app.dart</code>).

7.B Flutter — navigation, écrans et onglets

Fichier : flutter_app/lib/app.dart

- `App` : `MaterialApp` avec le thème `myTheme` (Material 3 sombre, défini dans `panache_theme.dart`).
- Pile d'écrans selon l'état (`Consumer2<AuthProvider, SecurityProvider>`) :
 1. `LoadingScreen` si `auth.isLoading` ;
 2. `UpdatePasswordScreen` si récupération de mot de passe Appwrite ;
 3. `EmailVerificationPendingScreen` si email pas encore vérifié ;
 4. `LoginScreen` si pas connecté ;
 5. `AppLockScreen` si PIN / biométrie active et session expirée ou app verrouillée ;
 6. sinon `HomeTabs`.

- **NodexOpeningSplash** : animation plein écran au premier lancement, puis `onFinished` enlève le splash.
 - **HomeTabs** : liste fixe de 5 pages — `DashboardScreen`, `WalletsScreen`, `HistoryScreen`, `CardScreen`, `SettingsScreen`. Transitions **FadeThroughTransition** entre onglets. Fond **FuturisticBackground**.
 - **FAB « Assistant »** : `Navigator.push` vers `ChatScreen`.
 - **WidgetsBindingObserver** : quand l'app passe en arrière-plan, si le PIN est activé → verrouillage ; au retour → contrôle d'expiration de session.
 - **Listener(onPointerDown)** : chaque toucher réinitialise le minuteur d'inactivité (`SecurityProvider.touchActivity()`).
-

7.C Flutter — authentification (Provider + service Appwrite)

Fichiers : `flutter_app/lib/providers/auth_provider.dart`, `flutter_app/lib/services/auth_service_appwrite.dart`

AuthServiceAppwrite encapsule le SDK Appwrite :

- `getCurrentUser()` : `account.get()` → objet local **User** (id, email, name).
- `getAccessToken()` : `account.createJWT()` → chaîne JWT courte durée (~**15 min**), renvoyée au backend Laravel dans **Authorization**.
- `login` / `register` / `OTP` / `magic link` / `mot de passe oublié` : appellent les méthodes Appwrite correspondantes.

AuthProvider :

- `_syncTokenToApi()` : après connexion, copie le JWT dans **ApiClient** (`setToken`) pour que **toutes** les requêtes Laravel portent le bon en-tête.
 - `checkAuth()` : au démarrage, tente de récupérer l'utilisateur ; sur le **web**, gère les URLs de callback `verify-email` et `magic URL` (`userId` + `secret` dans la query). Timeout 8 s + filet de sécurité 10 s pour ne pas bloquer indéfiniment sur l'écran de chargement.
 - `syncApiToken()` : à appeler avant les appels sensibles pour **rafraîchir** le JWT expiré.
 - `notifyListeners()` : déclenche le **rebuild** des widgets qui écoutent le provider (passage entre écran login et accueil).
-

7.D Flutter — sécurité locale (PIN, biométrie, journal)

Fichier : `flutter_app/lib/providers/security_provider.dart`

- Utilise **FlutterSecureStorage** pour le hash du PIN et **SharedPreferences** pour les options (timeout, biométrie, journal).
 - **SecurityEvent** : petit modèle JSON pour le journal (type, description, date).
 - Gestion du **verrouillage** (`isLocked`), du **délai d'inactivité** (`session_timeout_min`), option **biométrie**, nettoyage presse-papiers, tentatives de code PIN échouées (avec option `enableLoginCooldown` désactivée en démo).
-

7.E Flutter — client HTTP vers Laravel

Fichier : `flutter_app/lib/services/api_client.dart`

- **Pattern singleton** : `ApiClient._()` constructeur privé + `factory ApiClient() => _instance` → une seule instance partagée.
 - `_uri(path)` : si `path` commence par `http`, URL absolue; sinon concatène `ApiConfig.baseUrl` + chemin (ex. `/virements/send`).
 - `_getToken()` : 1) JWT Appwrite via `AuthServiceAppwrite.getAccessToken()`; 2) sinon token sauvegardé sous la clé `jwt_token` dans `SharedPreferences` (secours).
 - `get / post` : en-têtes JSON + Bearer; **timeout 15 s**. Le code **ne supprime pas** le token sur 401 : la session Appwrite peut encore être valide alors que le JWT Laravel est rejeté → l'utilisateur peut se **resynchroniser** avec `syncApiToken()`.
-

7.F Flutter — URL de l'API

Fichier : `flutter_app/lib/config/api_config.dart`

- **Priorité** : 1) URL saisie dans Réglages (clé `nodex_api_base_url`); 2) `--dart-define=API_BASE_URL=...` à la compilation; 3) défaut plateforme : `127.0.0.1:8000/api` (iOS, desktop, web hors hébergement), `10.0.2.2:8000/api` sur émulateur Android (alias de la machine hôte).
 - `_normalizeApiBase` : enlève le `/` final et **force** le suffixe `/api` pour que les chemins relatifs (`/wallets`) soient corrects.
-

7.G Flutter — portefeuilles, cours, transactions

Fichier : `flutter_app/lib/providers/wallet_provider.dart`

- **Classes locales** :
 - `Wallet` : id, blockchain, adresse, balance, symbole, nom, icône.
 - `Transaction` : type (`send`, `receive`, `swap`, `buy`, `bank_send`, `bank_receive`), montant, description, date, etc.
 - `_chainMeta` : pour ETH, SOL, ALGO, BTC — nom affiché, lettre d'icône, `cgId` CoinGecko pour l'API prix.
 - `_soldeDepartEur (2000)` : valeur affichée tant que le solde serveur n'est pas chargé (UX démo).
 - `fetch(userId)` (schéma) : charge `GET /wallets`, `GET /virements/me`, `GET /virements/history`, `GET /virements/balance`, appelle `_fetchPricesFromCoinGecko`, fusionne virements dans `_transactions` via `_mergeVirementsIntoTransactions`, met à jour `notifyListeners()`.
 - `_fetchPricesFromCoinGecko` : `GET https://api.coingecko.com/api/v3/simple/price?ids=...&vs_currencies=eu` → remplit `_prices` et `_changes24h`. En cas d'erreur réseau, les maps restent vides (pas de faux cours).
 - `totalBalanceEur` : somme du solde EUR + Σ (balance crypto $\times$ prix).
 - `testConnection()` : diagnostic `GET /health` puis `GET /virements/me` pour l'écran Réglages.
-

7.H Flutter — assistant conversationnel

Fichier : `flutter_app/lib/services/chat_service.dart`

- Prompt système fixe : rôle « assistant NodEX », français, **ne pas inventer** les chiffres du compte.

- Historique `_history` limité aux **16 derniers** messages pour ne pas dépasser les limites de tokens.
 - **Flux principal** : `POST /chat/groq` avec `messages` (system + historique).
 - **Secours** : si 401 → Groq direct avec contexte local ; si 503 (pas de clé serveur) → Groq direct ; si erreur réseau → Groq direct si clé présente dans l'app ; sinon message invitant à configurer Laravel ou la clé dans Réglages.
-

7.I Laravel — enregistrement des middlewares API

Fichier : `backend-laravel/bootstrap/app.php`

- Les routes définies dans `routes/api.php` passent par la pile `api`, à laquelle sont **préfixés** (prepend) :
 1. **CorsAllowLocalhost** — réponses CORS pour Flutter **web** sur localhost ;
 2. **EnsureNodexUser** — crée un enregistrement `NodexUser` si le JWT Appwrite est valide mais l'utilisateur n'existe pas encore en base.
-

7.J Laravel — CORS localhost

Fichier : `backend-laravel/app/Http/Middleware/CorsAllowLocalhost.php`

- Si l'en-tête `Origin` est `http(s)://localhost` ou `127.0.0.1`, ajoute `Access-Control-Allow-Origin` avec cette origine exacte, `Allow-Credentials`, et gère la requête **OPTIONS** (preflight) en **204** sans corps. Sans cela, le navigateur bloquerait les requêtes avec `Authorization` depuis une autre « origine » (port différent).
-

7.K Laravel — création automatique du compte NodEX

Fichier : `backend-laravel/app/Http/Middleware/EnsureNodexUser.php`

- Lit le JWT Bearer, décode le payload (même logique que `ResolvesNodexUser`).
 - Si `NodexUser` avec cet `appwriteId` n'existe pas : crée une ligne avec `firstOrCreate` : UUID interne, email technique `{appwriteId}@nodex-local.invalid`, nom par défaut, **pseudonyme** et **IBAN synthétiques** (méthodes du modèle), `balanceEur` initial **2000** (démonstration).
 - Puis laisse passer la requête vers le contrôleur. Ainsi la première requête authentifiée « provisionne » le compte côté Laravel.
-

7.L Laravel — résolution utilisateur dans les contrôleurs

Fichier : `backend-laravel/app/Http/Controllers/Concerns/ResolvesNodexUser.php`

- `appwriteIdFromJwt` : extrait la partie centrale du JWT (base64 URL-safe), `json_decode`, lit `userId` / `sub` / `$id` ou `user.$id`.
 - `nodexUser` : `NodexUser::where('appwriteId', $id)->first()` ou `null`.
 - **Utilisé par** : `WalletController`, `CardController`, `VirementController`, `ChatController` — même règle partout.
-

7.M Laravel — VirementController (détail de send)

Fichier : `backend-laravel/app/Http/Controllers/VirementController.php`

1. Résout l'expéditeur via JWT → `NodexUser`.
2. Lit `toIdentifier` (ou ancien `toEmail`) et `amount`.
3. Valide : destinataire non vide, montant > 0 , solde **au moins égal au** montant demandé.
4. **Recherche du destinataire** :
 - si la saisie ressemble à un IBAN FR (normalisé sans espaces), parcourt les utilisateurs avec IBAN et compare ;
 - sinon recherche par **pseudonym** (insensible à la casse, `LOWER(pseudonym)`) ;
 - sinon si la chaîne contient @, recherche par **email**.
5. Vérifie que le destinataire n'est pas soi-même.
6. `DB::transaction` : met à jour `balanceEur` des deux lignes `NodexUser`, insère une ligne `VirementEur` (`fromUserId, toUserId, amount`).
7. Réponse JSON : `success, newBalance, recipientNewBalance`.

Les méthodes **balance**, **me**, **history** sont en lecture seule : solde, infos RIB, liste fusionnée des virements avec pseudonyme de la contrepartie.

7.N Laravel — WalletController

Fichier : `backend-laravel/app/Http/Controllers/WalletController.php`

- Tableau constant **CHAINS** : ETH, BTC, SOL, ALGO avec noms et blockchains.
 - Pour chaque symbole : `balance` lue dans le `cache` Laravel `nodex_crypto_{userId}` ou 0 ; `address` = préfixe chaîne + tronquat de `hash('sha256', userId + symbol)` pour une adresse **reproductible** (démon, pas une vraie dérivation HD wallet).
-

7.O Laravel — CardController et algorithme de Luhn

Fichier : `backend-laravel/app/Http/Controllers/CardController.php`

- `index` : `UserCard` pour ce `userId` ; sinon `createCard`.
 - Génère un numéro de 16 chiffres dont les 15 premiers dépendent de l'utilisateur, puis calcule le **chiffre de contrôle Luhn** (`luhnCheckDigit`) pour que le numéro soit valide au même titre qu'une carte test.
 - Stocke `last4`, mois/année d'expiration, CVV, PIN ; renvoie JSON pour l'app (masquage côté UI sauf après action « voir »).
-

7.P Laravel — ChatController (Groq + contexte)

Fichier : `backend-laravel/app/Http/Controllers/ChatController.php`

- Vérifie `GROQ_API_KEY` dans la config / `.env` (sinon 503).
 - `buildAccountFactsBlock` : texte structuré avec solde EUR, nom, pseudo, IBAN, lignes crypto, **8 derniers virements** avec libellés envoyé/reçu.
 - `injectLiveAccountContext` : ajoute ce texte au message **system** (ou crée un system en tête de liste).
 - Appel HTTP serveur à serveur vers **Groq** (`chat/completions`) ; renvoie le JSON Groq tel quel au client en 200, ou message d'erreur en 502.
-

7.Q Fichiers de configuration (sans logique métier)

- `flutter_app/lib/config/appwrite_config.dart` : endpoint Appwrite, projet, IDs (non commités en clair dans le dépôt public — utiliser des placeholders / `.env` selon le setup).
 - `flutter_app/lib/config/groq_config.dart` / `stripe_config.dart` : modèles et clés côté client où autorisé.
 - `backend-laravel/config/services.php` : clé Groq et nom de modèle lus par `ChatController`.
 - `backend-laravel/routes/api.php` : déclare toutes les routes listées en section 6.
-

8. Méthode et mise en œuvre

1. Mise en place du backend Laravel (`.env`, Composer, migrations, `php artisan serve`).
 2. Développement de l'app Flutter et connexion à l'API (`API_BASE_URL` via `--dart-define` ou réglages in-app).
 3. Configuration Appwrite et variables sensibles **hors dépôt** (fichier `.env` ignoré par Git).
 4. Tests manuels sur simulateur iOS / appareil réel (réseau local, URL du Mac).
 5. Versionnement Git et push sur la branche **meranville**.
-

9. Ce qui manque pour un déploiement « grand public »

Ce projet est adapté au **développement et à la démonstration**. Pour une **publication large** (stores, utilisateurs inconnus, charge réelle), il faudrait notamment :

9.1 Produit et conformité

- **Statut légal** : services de paiement, crypto et « banking » sont **réglementés** (ex. Europe : DSP2, AML, licences selon l'activité). Un prototype étudiant **ne remplace pas** un cadre juridique ni un établissement agréé.
- **CGU / politique de confidentialité / RGPD** : textes validés, base légale du traitement des données, droits utilisateurs, DPO si nécessaire.
- **Mentions « démo / non régulé »** si l'app reste un projet pédagogique accessible publiquement.

9.2 Sécurité

- Supprimer **/api/debug/auth** et tout endpoint de debug en production.
- **HTTPS** obligatoire partout (API, Appwrite, callbacks).
- Secrets uniquement en variables d'environnement **serveur** ; rotation des clés ; principe du moindre privilège.
- Revue des en-têtes CORS, limitation du débit (**rate limiting**), validation **cryptographique** du JWT (au-delà du simple décodage base64 utilisé pour résoudre l'utilisateur — en production il faut vérifier la signature avec les clés Appwrite).
- Audit de sécurité (OWASP Mobile / API) et tests d'intrusion pour un vrai lancement.

9.3 Infrastructure

- Hébergement **géré** (serveur, base de données, sauvegardes, monitoring, logs centralisés).
- **CI/CD** (tests automatiques, build signé iOS/Android).
- **Scalabilité** (file d'attente pour tâches longues, cache si besoin).

9.4 Qualité et maintenance

- **Tests automatisés** (unitaires, intégration API, tests widget Flutter).
- Gestion des **versions**, changelog, canal de **support** utilisateur.
- **Accessibilité** (a11y) et **internationalisation** (i18n) si public multilingue.

9.5 Stores (Apple App Store, Google Play)

- Comptes développeur payants, fiches conformes aux guidelines, captures d'écran, politique de confidentialité hébergée en URL publique.
- Déclaration des **achats intégrés** / Stripe selon les règles de chaque store.
- Build **signé**, provisioning iOS, Android App Bundle.

9.6 Opérations financières réelles

- Compte **Stripe** (ou autre PSP) en mode **live**, webhooks sécurisés, conformité **PCI** (ne jamais stocker PAN en clair).
- KYC / lutte contre la fraude si encaissement ou comptes utilisateurs réels.

En résumé : le travail actuel constitue une **base technique solide pour un projet universitaire** ; le passage au **grand public** implique juridique, sécurité, exploitation et qualité logicielle bien au-delà du périmètre d'un cours.

10. Difficultés rencontrées (exemples types)

- Faire coïncider l'**URL de l'API** entre simulateur, téléphone physique et machine hébergeant Laravel (adresse IP locale, pare-feu).
- Gérer l'**authentification** (JWT Appwrite) côté Laravel et côté Flutter de manière cohérente.

- Comprendre la **structure du dépôt** (Flutter dans un sous-dossier, pas à la racine).
- Configurer **Stripe** et **Appwrite** sans commiter de secrets (`.env`, `.gitignore`).
- Limiter les **hallucinations** de l'IA sur les soldes grâce au **contexte injecté** côté Laravel.

(À adapter avec mes propres anecdotes pour l'oral.)

11. Conclusion

NodEX regroupe une **application Flutter**, une **API Laravel**, **Appwrite**, **Groq**, **CoinGecko** et **Stripe** dans une expérience de portefeuille et de services associés. Le rendu est **démontrable en local**, illustré par des **captures d'écran** dans ce rapport, et documenté dans le `README.md` du dépôt. Pour une **diffusion publique**, il reste indispensable de traiter **légal**, **sécurité**, **hébergement**, **stores** et **exploitation** comme un produit à part entière.

12. Sources, documentation et ressources utilisées

Les liens ci-dessous sont les **sources officielles ou principales** qui m'ont aidé. Les URL sont données en entier pour faciliter la vérification.

12.1 Frameworks et langages

Ressource	URL
Flutter — documentation	https://docs.flutter.dev/
Dart — langage	https://dart.dev/
Laravel — documentation	https://laravel.com/docs
PHP	https://www.php.net/docs.php
Composer (PHP)	https://getcomposer.org/doc/

12.2 Services et API externes du projet

Ressource	URL
Appwrite — documentation	https://appwrite.io/docs
Stripe — documentation développeurs	https://stripe.com/docs
Stripe — Flutter SDK (flutter_stripe)	https://pub.dev/packages/flutter_stripe
Groq — console / clés API	https://console.groq.com/
Groq — documentation modèles API	https://console.groq.com/docs/models
CoinGecko — documentation API (prix crypto)	https://www.coingecko.com/en/api/documenta

12.3 Packages Flutter (référence pub.dev)

Package	URL
provider	https://pub.dev/packages/provider
http	https://pub.dev/packages/http
appwrite (SDK Dart)	https://pub.dev/packages/appwrite
flutter_secure_storage	https://pub.dev/packages/flutter_secure_storage
local_auth	https://pub.dev/packages/local_auth
shared_preferences	https://pub.dev/packages/shared_preferences
qr_flutter	https://pub.dev/packages/qr_flutter
animations	https://pub.dev/packages/animations

Package	URL
flutter_lints	https://pub.dev/packages/flutter_lints

12.4 UI / design

Ressource	URL
Material Design 3	https://m3.material.io/
Panache (thème Flutter, référence dans le code)	https://github.com/rxlabz/panache

12.5 Outils, versionnement, présentation

Ressource	URL
Git — documentation	https://git-scm.com/doc
GitHub	https://github.com/
Gamma (présentations assistées par IA, si utilisé)	https://gamma.app/

12.6 Sécurité et bonnes pratiques (références générales)

Ressource	URL
OWASP Mobile Top 10	https://owasp.org/www-project-mobile-top-10/
CNIL — RGPD (information grand public)	https://www.cnil.fr/fr/rgpd-de-quoi-parle-t-on

12.7 Dépôt du cours / du projet

Ressource	URL
Dépôt GitHub mobileapp-2026	https://github.com/roor-killla/mobileapp-2026

13. Déclaration d'honnêteté académique

Je déclare que ce rapport décrit mon travail et que les sources externes sont citées dans la section 12. Les captures d'écran proviennent de **mon** exécution de l'application NodEX. Les extraits de code tiers respectent les licences des projets concernés.